AIX-MARSEILLE UNIVERSITÉ
MASTER 2 SID
Apprentissage automatique pour la science des données
Année universitaire 2026-2027

TD n°1 : Recherche de motifs

Par Muriel Visani

Exercice 1 : Données tabulaires

Considérons les données tabulaires suivantes, avec : min_sup = 2/5 = 40%/

Transaction	Items
T_1	A, B, C
T_2	A, B, D
T_3	A, B, E
T_4	C, D, E
T_5	C, D, E

1. À l'aide de l'algorithme Apriori, donner la liste des itemsets fréquents.

2. À l'aide de l'algorithme CLOSE, donner la liste des itemsets **fermés** fréquents.

3. On fixe maintenant un seuil minimal de confiance pour retenir les règles d'association : minconf=100%. Quelles sont les règles obtenues avec l'algorithme Apriori ?

4. Toujours avec une valeur de minconf=100%, quelles sont les règles d'association que l'on peut générer à partir des itemsets fermés fréquents donnés par CLOSE ?

5. Même question, mais avec CLOSE et une minconf de 50%

Exercice 2 :
On veut maintenant appliquer l'algorithme FP-Growth à partir des données suivantes :

T_ID	**Items**
T_1	A, E
T_2	A, D
T_3	A, B, E
T_4	C, D, E
T_5	C, D, E

1. Construire, à la main (et pas sur ordinateur), l'arbe FP-Tree à partir de ces données, et d'un min_sup = 2/5.

2. Je veux appliquer l'algorithme FP-Growth à partir de ces données, et d'un min_sup = 2/5, pour obtenir les itemsets fréquents. J'ai demandé à une célèbre IA de le faire pour

moi. Voici le résultat (très détaillé), dans lequel il y a plusieurs erreurs assez grossières.

a. Identifiez les erreurs commises par cette IA sur cet exemple : d'où proviennent ces erreurs ?
b. Il semblerait que l'IA se soit rendu compte que quelque chose n'allait pas, et qu'elle ait inventé une règle : quelle est cette règle ?
c. En quoi cette règle est-elle fausse ?

📌 Étape 1 : Mining pour `C` (support=2)

1. Trouver les suffix paths pour `C` :

- `C` apparaît **une fois** dans l'arbre (`C:2`).
- **Chemin complet** depuis la racine : `ø → E:4 → D:2 → C:2` → **Suffix path** : `<(E:4, D:2), C:2>`

2. Construire le FP-tree conditionnel pour `C` :

- **Préfixe** du suffix path : `(E:4, D:2)`
- On crée un nouvel arbre avec ces items et leurs compteurs :

```
text                                                    Copier
ø
└── E:4
    └── D:2
```

- **Vérification des supports** :
  - `E:4` ≥ 2 (OK)
  - `D:2` ≥ 2 (OK) → **Aucun filtre nécessaire**.

3. Extraire les motifs fréquents avec `C` :

- **Motifs dans l'arbre conditionnel** :
  - `ø` (support=2, car `C:2` apparaît 2 fois)
  - `E` (support=4, mais dans le contexte de `C`, on prend le **minimum** entre le support dans l'arbre conditionnel et celui de `C` → **min(4,2)=2**)
  - `D` (support=2)
  - `E,D` (support=2)
- **Combinaison avec `C` :**
  - `{C}` (support=2)
  - `{E,C}` (support=2)
  - `{D,C}` (support=2)
  - `{E,D,C}` (support=2)

4. Motifs fréquents trouvés :

`{C}, {E,C}, {D,C}, {E,D,C}` (tous avec support=2).

📌 Étape 2 : Mining pour `D` (support=3)

1. Trouver les suffix paths pour `D` :

- `D` apparaît **deux fois** dans l'arbre :
  - `D:2` (sous `E:4`)
  - `D:1` (sous `A:1`)
- **Chemin 1** : `ø → E:4 → D:2` → **Suffix path** : `<(E:4), D:2>`
- **Chemin 2** : `ø → A:1 → D:1` → **Suffix path** : `<(A:1), D:1>`

2. Construire le FP-tree conditionnel pour `D` :

- **Préfixes** :
  - `(E:4)` (pour `D:2`)
  - `(A:1)` (pour `D:1`)
- **Fusion** :
  - On crée un nouvel arbre avec :
    - `E:4` (depuis le 1er chemin)
    - `A:1` (depuis le 2ème chemin)
  - **Arbre conditionnel** :

```text
ø
├── E:4
└── A:1
```

- **Vérification des supports** :
  - `E:4` ≥ 2 (OK)
  - `A:1` < 2 (**à filtrer** car min_support=2) → **On élimine** `A:1` (son support dans l'arbre conditionnel est 1 < 2).
  - **Arbre conditionnel filtré** :

```text
ø
└── E:4
```

3. Extraire les motifs fréquents avec `D` :

- **Motifs dans l'arbre conditionnel filtré** :
  - `ø` (support=3, car `D` apparaît 3 fois au total)
  - `E` (support=4, mais dans le contexte de `D`, on prend **min(4,3)=3**)
- **Combinaison avec `D`** :
  - `{D}` (support=3)
  - `{E,D}` (support=3)

4. Motifs fréquents trouvés :

`{D}`, `{E,D}` (supports 3 et 3).

AIX-MARSEILLE UNIVERSITÉ
MASTER 2 SID
Apprentissage automatique pour la science des données
Année universitaire 2026-2027

📌 Étape 3 : Mining pour `A` (support=3)

1. Trouver les suffix paths pour `A` :

- `A` apparaît **deux fois** dans l'arbre :
  - `A:2` (sous `E:4`)
  - `A:1` (direct sous racine)
- **Chemin 1** : `ø → E:4 → A:2` → **Suffix path** : `<(E:4), A:2>`
- **Chemin 2** : `ø → A:1` → **Suffix path** : `<, A:1>`

2. Construire le FP-tree conditionnel pour `A` :

- **Préfixes** :
  - `(E:4)` (pour `A:2`)
  - `ø` (pour `A:1`)
- **Fusion** :
  - On crée un nouvel arbre avec :
    - `E:4` (depuis le 1er chemin)
  - **Arbre conditionnel** :

```
text                                                    Copier
ø
└── E:4
```

- **Vérification des supports** :
  - `E:4` ≥ 2 (OK) → **Aucun filtre**.

3. Extraire les motifs fréquents avec `A` :

- **Motifs dans l'arbre conditionnel** :
  - `ø` (support=3, car `A` apparaît 3 fois au total)
  - `E` (support=4, mais dans le contexte de `A`, on prend **min(4,3)=3**)
- **Combinaison avec `A` :**
  - `{A}` (support=3)
  - `{E,A}` (support=3)

4. Motifs fréquents trouvés :

`{A}`, `{E,A}` (supports 3 et 3).

AIX-MARSEILLE UNIVERSITÉ
MASTER 2 SID
Apprentissage automatique pour la science des données
Année universitaire 2026-2027

📌 Étape 4 : Mining pour `E` (support=4)

1. Trouver les suffix paths pour `E` :

- `E` apparaît **une fois** dans l'arbre (`E:4`).
- **Chemin** : `∅ → E:4` → **Suffix path** : `<, E:4>`

2. Construire le FP-tree conditionnel pour `E` :

- **Préfixe** : `∅` (pas d'items avant `E`).
- **Arbre conditionnel** : vide (seul `E` est présent).
- **Vérification des supports** :
  - Aucun item à vérifier.

3. Extraire les motifs fréquents avec `E` :

- **Motifs dans l'arbre conditionnel** :
  - `∅` (support=4)
- **Combinaison avec `E`** :
  - `{E}` (support=4)

4. Motifs fréquents trouvés :

`{E}` (support=4).

✅ Liste complète des motifs fréquents (support ≥ 2)

Voici **tous les motifs fréquents** extraits, avec leur support :

Motif	Support	Source (mining de)
`{E}`	4	E
`{D}`	3	D
`{A}`	3	A
`{C}`	2	C
`{E,D}`	3	D
`{E,A}`	3	A
`{E,C}`	2	C
`{D,C}`	2	C
`{E,D,C}`	2	C

> **Remarque** :
>
> - `{A,D}` n'apparaît **pas** dans la liste car son support est **1** (seulement la transaction `T₂: [A,D]` le contient, mais après filtrage et fusion, son support dans l'arbre conditionnel de `A` ou `D` était insuffisant).
> - `{A,C}` n'apparaît **pas** car il n'existe **aucun chemin** dans l'arbre où `A` et `C` coexistent (ils ne sont jamais dans la même transaction après filtrage).

3. **Sur machine**, appliquer l'algorithme FP-Growth aux données de ce TD.

Indices :

- L'algorithme FP-growth est inclus dans la librairie mlxtend
- Le pré-traitement nécessaire pour les données, avant de les donner à l'algorithme FP-Growth, est de les mettre sous la forme de booléens en colonnes (entre autres, la méthode TransactionEncoder de mlxtend.preprocessing fait cela)